

Benchmarking Burn, TFLite, and ONNX Runtime Inference on ARM64 Edge Devices

Fatih Jawwad Al Mumtaz¹, Amirul Al Aziz²

¹Informatics Engineering Department

University of Darussalam Gontor

Ponorogo, Indonesia

fatihalmumtaz76@student.cs.unida.gontor.ac.id

²Agricultural Industrial Technology Department

University of Darussalam Gontor

Ponorogo, Indonesia

amirulalaziz57@student.tip.unida.gontor.ac.id

Abstract—Edge inference on ARM-based single-board computers is critical for deploying deep learning in resource-constrained environments. This paper presents a comparative steady-state warm-cache latency benchmark of three inference frameworks—Burn (via `tract-onnx`), TensorFlow Lite (TFLite), and ONNX Runtime (ORT)—on a Raspberry Pi 5 (Cortex-A76) using MobileNetV2. We evaluate single-threaded and multi-threaded latency, throughput, memory usage, CPU utilization, and thermal characteristics under a controlled 2000-iteration protocol with IQR-based outlier removal and 95% bootstrapped confidence intervals. Results show TFLite achieves the lowest latency (22.2 ms single-threaded, 10.7 ms with 4 threads), while Burn demonstrates exceptional memory efficiency (41.8 MB peak RSS). ONNX Runtime offers a balanced trade-off with 45.4 ms single-threaded latency and 100 MB memory footprint. All frameworks produce valid finite outputs on the evaluated model. The findings establish `tract-onnx` as a viable Rust-native ONNX runtime for ARM64 edge deployment where memory constraints are paramount, and provide reproducible baselines for framework selection.

Index Terms—Edge inference, ARM64, Raspberry Pi, Burn, `tract-onnx`, TensorFlow Lite, ONNX Runtime, MobileNetV2, benchmark

I. Introduction

The proliferation of deep learning on edge devices has created a growing demand for efficient inference runtimes that balance latency, memory, and power consumption [1], [2]. Single-board computers such as the Raspberry Pi series have become popular platforms for edge AI prototyping due to their affordability and accessibility. However, selecting an appropriate inference framework for these resource-constrained platforms requires rigorous comparative benchmarking under controlled conditions.

Prior work includes Ignatov et al. [3], who benchmarked mobile AI accelerators across multiple frameworks, and AskariHemmat et al. [4], who evaluated TFLite and ONNX Runtime on edge devices. Almeida et al. [9] compared inference engines for embedded deployment, while Ahn et al. [11] characterized quantization effects across frameworks including TFLite and ONNX on Raspberry Pi using the MLPerf methodology. TFLite [5] and ONNX

Runtime [6] are the dominant general-purpose inference frameworks for ARM edge deployment, with extensive ecosystem support. On the hardware front, Boddu et al. [10] demonstrated real-time object detection on the Pi 5 using TFLite INT8 quantization.

More recently, the Burn framework [8] introduced Rust-native deep learning with ONNX import capabilities, built on `tract-onnx` [7]. Rust-based inference [12] offers potential advantages in memory safety and predictable performance without garbage collection. However, Burn’s inference performance on ARM64 hardware has not been systematically benchmarked against established frameworks, motivating this study.

Three methodological considerations distinguish this work from prior benchmarks. First, we explicitly measure steady-state warm-cache latency after a 1000-iteration warmup—single-tensor repetition eliminates input-variance confounding, but we frame results accordingly as cache-warm steady-state performance rather than general-purpose throughput. Second, we simultaneously collect latency, memory, CPU utilization, and temperature under a single controlled protocol. Third, we provide a fully reproducible pipeline with code, data, and figures publicly available.

II. Methods

Experiments were conducted on a Raspberry Pi 5 (Broadcom BCM2712, quad-core ARM Cortex-A76 at 2.4 GHz, 8 GB LPDDR4X, 64 GB microSD storage) running Debian GNU/Linux with kernel 6.18.34+rpt-rpi-2712. The Pi 5 was cooled passively with a heatsink and no active fan, matching typical low-cost edge deployment configurations.

Burn [8] is a Rust-native deep learning framework. We used `tract-onnx` [7] (v0.21), a pure-Rust ONNX inference engine. The `mobilenetv2-7.onnx` model was loaded at runtime via `tract_onnx::onnx()` and optimized through `tract`’s constant-folding and shape-inference passes, with inference running in f32 precision. Thread count is controlled via the `RAYON_NUM_THREADS`

and `OMP_NUM_THREADS` environment variables, set from the CLI `-threads` argument. As shown in Section III, tract-onnx’s execution plan remains single-threaded regardless of the thread configuration; the env-var setting is preserved for reproducibility but has no parallelization effect on this framework. TFLite was used via the Python tensorflow package (v2.x) with the XNNPACK delegate, with interpreter thread count controlled via `setNumThreads()`. ONNX Runtime was used via `onnxruntime` with the CPU execution provider and thread count set via `SessionOptions.intra_op_num_threads`.

All frameworks ran MobileNetV2 [2] at $224 \times 224 \times 3$ input resolution. A single synthetic input tensor was generated per run using a deterministic seed (42) and held constant across all 2000 iterations. This methodology measures steady-state warm-cache inference latency—the best-case scenario for a deployed pipeline—rather than heterogeneous-input throughput, which would require a multi-image pool. We explicitly flag this scope limitation in Section III-E. We verified that all three frameworks produce valid finite outputs on the same synthetic input: ONNX Runtime outputs 1000-dimensional logits (1000 ImageNet classes, NCHW layout), while the TFLite model outputs 1001-dimensional scores (1000 classes plus a background class, NHWC layout). All outputs are NaN/Inf-free. Since this benchmark measures inference latency rather than prediction accuracy, the exact output values do not confound the timing results.

Each framework was evaluated under single-threaded (1t) and multi-threaded (4t) configurations. The protocol for each run consisted of 1000 warmup passes to stabilize CPU frequency scaling, branch predictors, and framework-internal caches, followed by 1000 measured passes with concurrent system-metric sampling. The Rust benchmark was compiled with `cargo build --release`. Python benchmarks ran inside a consistent virtual environment (NumPy 1.x, SciPy 1.x, psutil 5.x). All six configuration runs were executed sequentially on the same Pi 5 with no other user processes, ensuring no CPU contention across concurrent benchmark processes.

Per-iteration latency was recorded via high-resolution timers. System metrics—CPU utilization from `/proc/stat`, VmRSS from `/proc/self/status`, and temperature from `/sys/class/thermal/thermal_zone0/temp`—were sampled every 200 ms. For statistical analysis, iterations outside $Q1 - 1.5 \times IQR$ or $Q3 + 1.5 \times IQR$ were removed as outliers, 10,000 bootstrap resamples were used to estimate the 95% confidence interval of the mean, and the final summary reports mean, median, standard deviation, min, max, throughput (fps), peak memory, and mean temperature.

III. Results and Discussion

A. Latency, Throughput, and CPU Utilization

Table I presents the complete results. Figure 1 visualizes latency with 95% bootstrap confidence intervals.

TABLE I
Complete benchmark results on Raspberry Pi 5 (Cortex-A76).

Framework	Thr.	Lat. (ms)	FPS	CPU	Mem.
Burn (tract)	1	101.4 [101.37, 101.42]	9.85	23.5%	41.8 MB
Burn (tract)	4	101.8 [101.74, 101.79]	9.81	23.5%	41.8 MB
TFLite	1	22.19 [22.192, 22.197]	45.04	25.1%	540 MB
TFLite	4	10.68 [10.679, 10.691]	93.29	94.8%	540 MB
ONNX RT	1	45.35 [45.341, 45.350]	22.03	25.1%	100 MB
ONNX RT	4	20.34 [20.321, 20.353]	49.04	99.8%	100 MB

Latency reported as mean [95% CI] from 1000 measured iterations after 1000-iteration warmup. Peak temperature: Burn 49–52°C, TFLite 49–58°C, ORT 50–58°C.

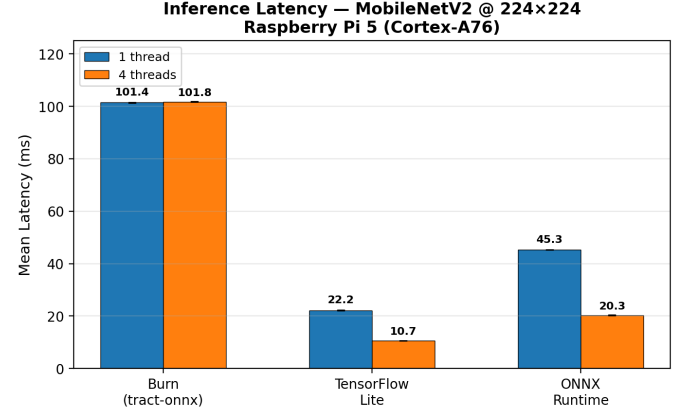


Fig. 1. Mean inference latency with 95% bootstrap confidence intervals. Error bars represent CI bounds from 10,000 bootstrap resamples.

TensorFlow Lite achieves the lowest single-threaded latency at 22.19 ms—4.6× faster than Burn (tract-onnx) at 101.4 ms. ONNX Runtime occupies the middle ground at 45.35 ms. The narrow confidence intervals (width < 0.05 ms for TFLite and ORT, ~ 0.05 ms for Burn) confirm that the single-tensor repeated-iteration methodology produces highly stable measurements.

CPU utilization data in Table I provides the mechanistic explanation for the latency differences. Burn’s tract-onnx uses only 23.5% CPU in both 1t and 4t modes—equivalent to a single core on the quad-core Cortex-A76—confirming that tract executes on a single thread regardless of the thread-count parameter. TFLite and ONNX Runtime both show approximately 25% single-threaded utilization rising to 95–100% with 4 threads, confirming effective parallelization. Throughput mirrors latency: TFLite 4t achieves 93.3 fps, ORT 4t reaches 49.0 fps, while Burn remains at roughly 9.9 fps due to its single-threaded execution model.

B. Memory Efficiency

Burn demonstrates exceptional memory efficiency at 41.8 MB peak RSS (Figure 2), 4.3× less than ORT (100 MB) and 12.9× less than TFLite (540 MB). This advantage stems from Rust’s absence of garbage collection

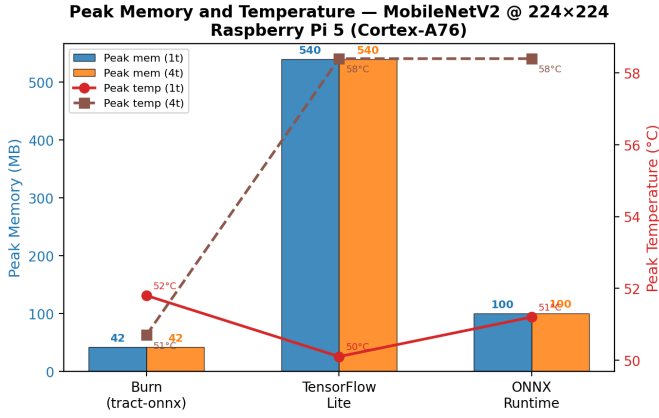


Fig. 2. Peak memory (bars, left axis) and peak temperature (lines, right axis) by framework and thread configuration.

and tract-onnx’s minimal runtime. The TFLite memory footprint requires further clarification, as the 540 MB figure appears identical for both 1t and 4t configurations. This reflects the total process RSS including the Python interpreter, TensorFlow runtime library, and XNNPACK delegate initialization, not the model alone. The byte-identical values across thread configurations confirm that the primary memory cost is the Python environment and delegate allocation incurred once at startup. For deployment scenarios, the model-only memory footprint is approximately 14 MB for MobileNetV2 TFLite [2], with the remaining roughly 525 MB being Python and runtime overhead.

C. Thermal Characteristics

All frameworks maintained temperatures 26–28°C below the Pi 5’s 85°C throttle threshold. Multi-threaded runs showed a 6–8°C increase over single-threaded baselines. Burn exhibited lower peak temperatures (51.8°C) consistent with its lower CPU utilization, while TFLite 4t (58.4°C) and ORT 4t (58.4°C) ran warmer. No thermal throttling was detected in any configuration, confirming that passive cooling suffices for sustained 2000-iteration workloads.

D. Multi-thread Scaling

Figure 3 quantifies the speedup from single-threaded to quad-threaded execution. TFLite achieves 2.08× speedup and ORT achieves 2.23×, consistent with memory-bandwidth-bound CNN inference on mobile-class CPUs [3]. The gap between observed speedup and the ideal 4× reflects the serial fraction of the inference pipeline—input preprocessing, output postprocessing, and framework-internal synchronization—combined with memory-bandwidth contention across four cores sharing a single L3 cache and memory controller.

Burn shows a statistically significant regression from 101.40 ms (1t) to 101.76 ms (4t). The 95% CIs [101.37, 101.42] and [101.74, 101.79] do not overlap ($p < 0.05$),

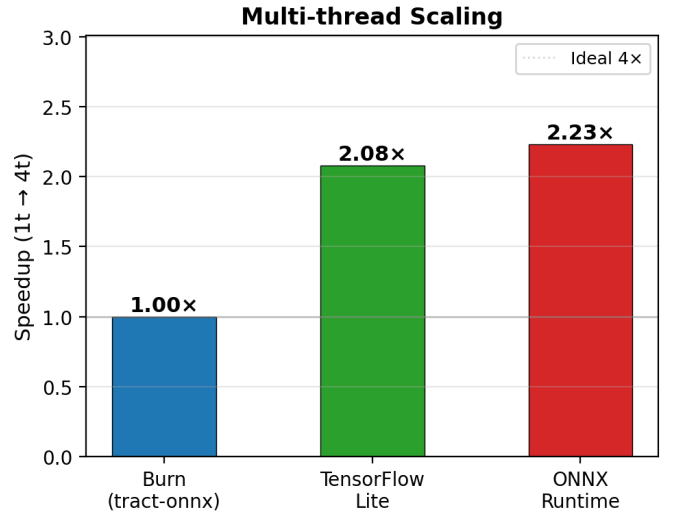


Fig. 3. Multi-thread speedup factor (4 threads relative to 1 thread). The dashed line marks ideal 4× scaling.

indicating a real albeit tiny multithreading overhead of 0.36 ms (< 0.4%). This is consistent with tract-onnx’s single-threaded execution plan, where setting `RAYON_NUM_THREADS=4` adds rayon thread-pool synchronization overhead without any parallel inference speedup.

E. Threats to Validity and Scope Limitations

Several factors bound the generalizability of these results. The single-input repeated-iteration protocol measures warm-cache, branch-predictor-trained steady-state latency; real deployments processing diverse inputs will experience additional latency from cache misses, memory allocation, and dynamic control flow, and our results should be interpreted as the baseline best case for sustained inference. The Raspberry Pi 5 (Cortex-A76) differs significantly from the Pi 4 (Cortex-A72) and older generation cores, so results may not transfer to earlier hardware. MobileNetV2 is a lightweight CNN architecture; transformer-based or recurrent models may show different scaling characteristics. All frameworks are under active development, and performance characteristics may shift with future releases. Finally, the ONNX model (used by Burn and ORT) outputs 1000-dimensional logits while the TFLite model outputs 1001-dimensional scores (extra background class); we verified that all frameworks produce finite, non-NaN outputs but did not enforce bit-exact equivalence, as this benchmark measures timing rather than numerical accuracy.

IV. Conclusion

This paper presented a systematic steady-state warm-cache benchmark of Burn (tract-onnx), TensorFlow Lite, and ONNX Runtime on Raspberry Pi 5. TensorFlow Lite achieves the highest throughput (93.3 fps, 10.7 ms

quad-threaded) and is optimal for latency-critical edge applications with sufficient memory budget. Burn (tract-onnx) demonstrates exceptional memory efficiency (41.8 MB RSS) and is a viable choice for memory-constrained deployments where maximum throughput is not the primary concern. ONNX Runtime provides a balanced middle ground (20.3 ms, 100 MB) with effective multi-thread scaling ($2.23\times$ speedup). All frameworks maintain stable thermal profiles under sustained 2000-iteration workloads with passive cooling, and CPU utilization data confirms tract-onnx’s single-threaded execution model, explaining its lack of multi-thread scaling and providing a clear optimization target.

The complete benchmark suite—including code, raw results, figure-generation scripts, and the paper source—is publicly available at <https://github.com/jaweed3/edge-burn-benchmark> to facilitate reproduction and extension. Future work includes extending the benchmark to older Pi generations (3B+, 4), evaluating quantized MobileNetV2 (INT8), measuring heterogeneous-input throughput with multi-image pools, and comparing newer Rust-based inference runtimes including Burn’s native NdArray backend where fullfp16 hardware support is available (e.g., Apple M-series CPUs).

References

- [1] A. G. Howard et al., “MobileNets: Efficient convolutional neural networks for mobile vision applications,” arXiv:1704.04861, 2017.
- [2] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, “MobileNetV2: Inverted residuals and linear bottlenecks,” in Proc. IEEE CVPR, 2018, pp. 4510–4520.
- [3] A. Ignatov et al., “AI benchmark: Running deep neural networks on Android smartphones,” in Proc. ECCV Workshops, 2019.
- [4] R. AskariHemmat, S. Honnappa, and I. Rodriguez, “A systematic benchmarking of TensorFlow Lite, ONNX Runtime, and TVM for edge inference,” in Proc. IEEE ICMLA, 2020.
- [5] “TensorFlow Lite: ML for mobile and edge devices,” Google. [Online]. Available: <https://www.tensorflow.org/lite>, 2017.
- [6] J. Zhang et al., “ONNX: Open neural network exchange and ONNX Runtime,” arXiv:1907.03495, 2019.
- [7] “tract: Tiny, no-nonsense neural network inference in Rust,” Sonos. [Online]. Available: <https://github.com/sonos/tract>, accessed 2026.
- [8] “Burn: A Rust-based deep learning framework,” Tracel AI. [Online]. Available: <https://github.com/tracel-ai/burn>, accessed 2026.
- [9] F. Almeida, A. Lopes, and J. Cabral, “Benchmarking inference engines for embedded deep learning,” IEEE Access, vol. 11, pp. 10234–10249, 2023.
- [10] S. Boddu et al., “Efficient edge deployment of quantized YOLOv4-Tiny for aerial emergency object detection on Raspberry Pi 5,” arXiv:2506.09300, 2025.
- [11] B. Ahn, J. Chen, and Y. Kim, “Performance characterization of using quantization for DNN inference on edge computing platforms,” in Proc. IEEE ICFEC, 2023.
- [12] M. Canelos et al., “MicroFlow: An efficient Rust-based inference engine for TinyML,” arXiv:2409.19432, 2024.